

```
class LunarRover(Vehicle):
    def __init__(self):
        print('LunarRover created.')
        self.engine = ElectricEngine()
```

This could require rewriting large amounts of code, particularly if you have several classes that inherit from a preexisting `Vehicle` class: all the `vehicleObj.changeSparkPlug()` calls would need to become `vehicleObj.engine.changeSparkPlug()` for every object of the `Vehicle` class or its subclasses. Because such a sizeable change could introduce bugs, you might want to simply have the `changeSparkPlug()` method for `LunarVehicle` do nothing. In this case, the Pythonic way is to set `changeSparkPlug` to `None` inside the `LunarVehicle` class:

```
class LunarRover(Vehicle):
    changeSparkPlug = None
    def __init__(self):
        print('LunarRover created.')
```

The `changeSparkPlug = None` line follows the syntax described in “Class Attributes” later in this chapter. This overrides the `changeSparkPlug()` method inherited from `Vehicle`, so calling it with a `LunarRover` object causes an error:

```
>>> myVehicle = LunarRover()
LunarRover created.
>>> myVehicle.changeSparkPlug()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'NoneType' object is not callable
```

This error allows us to fail fast and immediately see a problem if we try to call this inappropriate method with a `LunarRover` object. Any child classes of `LunarRover` also inherit this `None` value for `changeSparkPlug()`. The `TypeError: 'NoneType' object is not callable` error message tells us that the programmer of the `LunarRover` class intentionally set the `changeSparkPlug()` method to `None`. If no such method existed in the first place, we would have received a `NameError: name 'changeSparkPlug' is not defined` error message.